PROJECT SHEET 1 · NITISHVERIFIES / AXI4_UVM_VERIFICATION

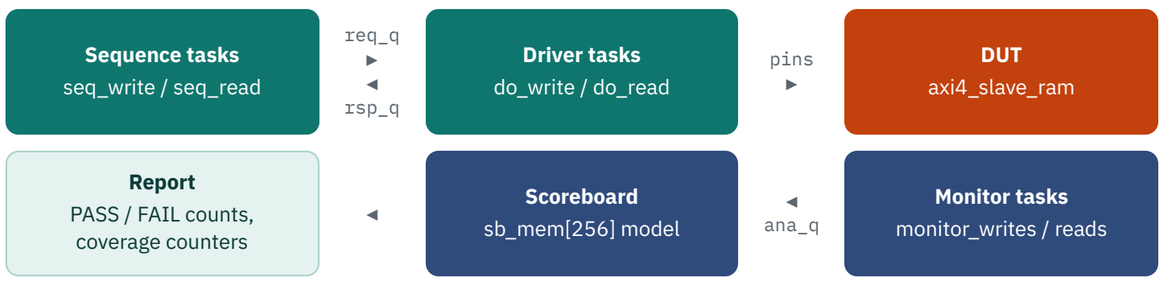
Your AXI4-Lite UVM Project, Explained

Know every file well enough to defend it. Interviewers at top silicon companies go deep on the project you list on your resume.

DUT: AXI4-Lite slave, 256 × 32-bit RAM
TB: task-based (runs on iverilog)
+ full UVM class library (tb/*.sv)

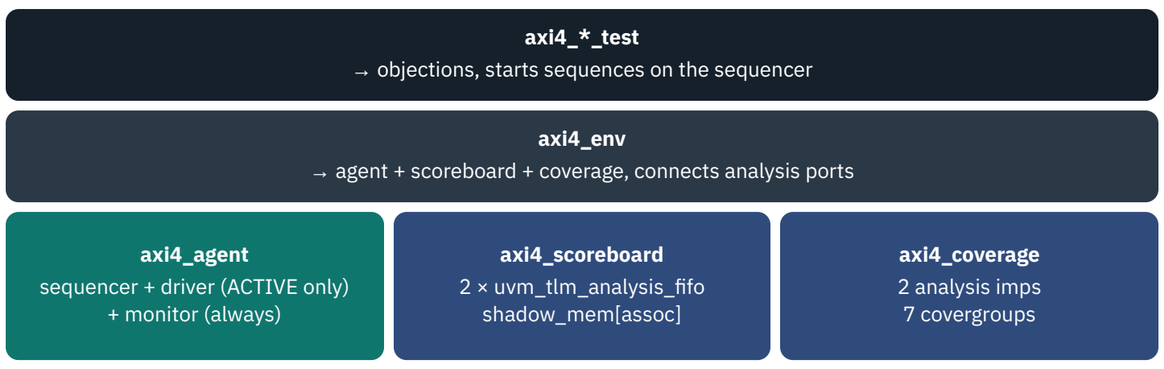
1 The repo has two testbenches — know which one actually runs

A · RUNNABLE: AXI4_TB_TOP.SV (TASK-BASED, IVERILOG)



- Hand-rolled FIFOs (flat arrays + head/tail/count + `wait(cnt>0)`) replace `mailbox`.
- Test picked by a compile-time `` `define `` from the Makefile.
- This is what `make all_tests` builds and runs.

B · UVM LIBRARY: TB/AXI4_*.SV (FOR VCS / QUESTA / XCELIUM)



Know this before you're asked: the Makefile never compiles these files. They are UVM-style code that hasn't been run yet — see the bugs sheet. Say so honestly, or fix it first.

2 UVM concept → where it lives in your repo

UVM concept	Task-based TB (A)	UVM library (B)
Transaction	`axi4_pkg::axi4_seq_item` (plain class)	`axi4_seq_item` extends uvm_sequence_item, rand + constraints, do_copy / do_compare
Sequence	seq_write / seq_read tasks	9 sequences: single, burst, rw, stress, error, wr_rd_check
Seq ↔ driver	req_q / rsp_q FIFOs	start_item / finish_item ↔ get_next_item / item_done
Driver	do_write: AW & W together, then B	Same, through `vif.driver_cb`
Monitor	Wait AW → W → B handshakes, push to ana_q	Forked write/read tasks → `ap_write`, `ap_read`
Analysis path	ana_q FIFO	analysis port → `uvm_tlm_analysis_fifo` (SB) and analysis imps (coverage)
Scoreboard	sb_mem[256], strobe-aware	assoc array shadow_mem, expects SLVERR ≥ 256 words
Coverage	Write / read counters	Address quadrant, data patterns, WSTRB, BRESP/RRESP
Config / VIF	Direct hierarchical access	`uvm_config_db::get(…"AXI4_VIF"…)`
Factory	—	`type_id::create` everywhere
End of test	`repeat(20) @(posedge)` + $finish	raise / drop objection + fixed #delay
Active / passive	—	`get_is_active()` in agent build_phase

3 The DUT: axi4_slave_ram

WRITE FSM (AS SHIPPED)



READ FSM



- **Single outstanding** per direction; reads and writes are independent FSMs.
- `apply_strobe()` merges new bytes into the old word using WSTRB.
- Index = `(addr >> 2) % 256` — out-of-range addresses **wrap** (alias), no SLVERR.
- RDATA is **registered**: it's stable while RVALID is held, which satisfies the handshake rule.
- Memory initialised to 0 in an `initial` block (simulation only — not reset logic).

4 Stimulus: the seq_item constraints

Constraint	What it does	Interview angle
`c_txn_type_dist`	60% writes, 40% reads	Why `:=` vs `:/` in dist
`c_addr_legal`	0x000–0x3FC, word aligned	How the error sequence overrides it with inline `with`
`c_wstrb`	Non-zero; 70% 1111, 5% each of 6 partial patterns	Only 7 of 15 legal patterns can ever occur
`c_prot`	Always 000	Hard constraint = no PROT coverage

Inline override: `axi4_error_seq` randomises with `addr inside {[0x400:0xFFFC]}` — but `c_addr_legal` still forbids that, so `randomize()` fails. Correct way: `txn.c_addr_legal.constraint_mode(0)` first, or put the range in a soft constraint.

5 Tests and what each really proves

Test	Stimulus	Really proves
write_test	8 writes, addr 0x00–0x1C	BRESP OKAY only — data not read back
read_test	8 reads after reset	Reset value 0
rw_test	Write then read, 8 addresses	Full-word data path
stress_test	32 random R/W, random strobes 1–15	Strobe merge + mixed traffic
error_test	Write/read 0xFFFF_FFFC	Nothing checked — aliasing hides it
wr_rd_chk_test	DEADBEEF, then CAFE0000 strb 1100	Expects 0xCAFE_BEEF

Every test drives AW and W in the **same cycle** with READY always accepted promptly. Real masters don't — that's the gap the bugs sheet exploits.

6 60-second project pitch

"I verified an **AXI4-Lite slave RAM** — 256 × 32 with byte strobes — using a **UVM-structured, self-checking** testbench: agent with driver, monitor and sequencer, a **scoreboard with a strobe-aware reference model**, and **functional coverage** on addresses, strobes and responses.

When I stressed the handshake ordering I found the DUT **deadlocks if write data arrives before the address**, that it **aliases out-of-range addresses** instead of returning SLVERR, and that my regression could **report PASS while the scoreboard logged errors**. I fixed all three and added assertions so they can't come back."

Only say the second paragraph **after** you've applied the fixes from the next sheet. Then it's true, and it's your strongest story.

7 Trace one write end to end

1. Test calls `seq_write(0x10, D)` → item pushed to `req_q`, sequence blocks on `rsp_q`.
2. Driver pops it, drives AWADDR/AWVALID + WDATA/WSTRB/WVALID on the next edge.
3. Slave (WR_ADDR) sees both handshakes in one cycle, writes `mem[4]`, raises BVALID.
4. Driver raises BREADY, captures BRESP, puts the item on `rsp_q` → sequence unblocks.
5. Monitor independently saw AW, W, B handshakes → builds its own item → `ana_q`.
6. Scoreboard applies strobes to `sb_mem[4]`, checks BRESP.

Key point: the scoreboard uses the **monitor's** item, never the driver's — so a driver bug can't hide a DUT bug.

8 Run it

```
sudo apt install iverilog gtkwave
cd sim
make sim TEST=axi4_rw_test SIM=iverilog
make all_tests SIM=iverilog
make waves TEST=axi4_rw_test
```

MAKEFILE ALSO TARGETS

- **Questa** — `vlog -uvm`, `vsim +UVM_TESTNAME`, UCDB coverage merge
- **VCS** — `-ntb_opts uvm`, `-cm line+cond+fsm+branch+tgl`, urg
- **Xcelium** — `xrun -uvm -coverage all`, imc
- Falls back to iverilog when the commercial tool isn't found.

Be able to explain `-sv_seed` / `+ntb_random_seed`: same seed reproduces the same random run — the basis of debugging a regression failure.